Lab 1.2 - Single Packet Authorization Execution

2026-11-04

Table of contents

1 Introduction **1**

2 Performing SPA for SSH Access **2**

3 Going Further (Optional) **6**

4 Clean Up **6**

1 Introduction

This lab is the second part of a brief tutorial for implementing Single Packet Authorization (SPA) with fwknop. Before completing anything in this lab, be sure to complete Lab 1.1 - Single Packet Authorization Setup.

Note that if you are doing this on a separate day than Lab 1.1, you may need to setup your variables again. On the client, re-set the following variables, replacing the x's with the appropriate values:

```
CLIENT_IP=138.47.x.x
SERVER_IP=138.47.x.x
SSH_PORT=22
```

You can confirm the variables are set with `echo $CLIENT_IP`, `echo $SERVER_IP`, and `echo $SSH_PORT`.

You will also set two variables on the **server** in this lab (`SSH_PORT` and `INTERFACE`). Variables only last for the terminal they were set in, so if you open a new terminal on the server at any point, including for the Clean Up section, set them again first. An empty variable can cause an `iptables` command to fail or to match the wrong rule.

2 Performing SPA for SSH Access

This portion will require you to go back and forth from client to server. As an overview, you will first test that the client can access the server via SSH. Then you will modify the firewall rules on the server to prevent all SSH connections and test it. Lastly, you will use fwknop on the client to send the SPA packets so that the firewall opens for the client to be able to ssh into it, but not anyone else.

1. On the client, test that you can ssh into the server. Be sure to use the appropriate username for the server.

   ```
   ssh -p $SSH_PORT username@$SERVER_IP
   ```

   This should successfully grant you access.

2. On the client, disconnect from the server with `ctrl+d`.

3. On the server, set a variable to represent the server's ssh port.

   ```
   SSH_PORT=22  # reminder: no spaces around = sign
   ```

4. On the server, get the interface for the network connection. This is the same interface you set as `PCAP_INTF` in Lab 1.1.

   ```
   ip -br addr
   ```

   Recall that the interface is the first column of the line containing your ip address, and is typically in the form `enp0s3` or `eth0`. Values may differ. Note what it is.

5. On the server, set another variable that holds the value for the interface (replace `enp0s3` with your interface).

   ```
   INTERFACE=enp0s3
   ```

6. On the server, view the current firewall rules to see the default with the `iptables` command. The flag -L is used to list the rules, the flag -n is used to output the numeric values for items that could resolve to non-numeric values (ex: show port 22 as `22` instead of as `ssh`), and the -v flag is to get extra details (verbose).

```
sudo iptables -L -nv
```

You should see information about the INPUT chain, the FORWARD chain, OUTPUT chain, and the FWKNOP_INPUT chain. INPUT, FORWARD and OUTPUT most likely have a policy of ACCEPT at this point.

Additionally, the INPUT chain should have 1 rule with FWKNOP_INPUT as the target.

7. On the server, add the following firewall rule with `iptables`. This rule is appending to the INPUT chain so that all ssh traffic is dropped.

```
sudo iptables -A INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
```

8. On the server, view the firewall rules again.

```
sudo iptables -L -nv
```

You should see that the INPUT chain has been modified with respect to port 22 and the interface you specified.

9. On the client, test ssh again. It should not allow you in. You can use `ctrl+c` to cancel the request when you think it has been too long. Be sure to use the appropriate username for the server.

```
ssh -p $SSH_PORT username@$SERVER_IP
```

10. On the client, use nmap to check if SSH is visible on the server.

```
sudo nmap -sS -p $SSH_PORT $SERVER_IP
```

It should show that the STATE is `filtered`. This means the probe received no reply at all, because the DROP rule silently discarded it. Compare this with `closed`, which nmap reports when the host replies that nothing is listening. A filtered port reveals less to an attacker.

11. On the client, use fwknop to send an SPA packet to the server.

```
fwknop -n $SERVER_IP
```

12. On the server, check that the SPA packet was accepted by checking the status of the fwknop server.

```
sudo systemctl status fwknop-server
```

You should see information concerning the acceptance of the SPA packet and a time limit for when it expires. For example, it may show the following:

```
Added access rule to FWKNOP_INPUT for x.x.x.x -> 0.0.0.0/0 tcp/22,
expires at 1746380800.
```

The expiration is a Unix timestamp (seconds since January 1, 1970). You can convert it to a readable date and time with `date -d @1746380800` (replace the number with yours). By default, the port is only opened for 30 seconds (the `FW_ACCESS_TIMEOUT` setting), so move quickly to the next step. You can view the temporary rule with `sudo iptables -L FWKNOP_INPUT -nv` before it expires.

Tip: to watch the fwknop log live instead of re-running `systemctl status`, open a second terminal on the server and run `sudo journalctl -u fwknop-server -f`. New messages appear as they arrive. Press `ctrl+c` to stop.

13. On the client, test ssh again to get access. Note that if the time limit expired, you will need to resend the SPA packet. Be sure to use the appropriate username for the server.

```
ssh -p $SSH_PORT username@$SERVER_IP
```

You should have access. Stay connected, and after the time limit expires, try typing a few commands. The session will freeze, because the temporary rule that let your packets through has been removed and the DROP rule now applies to them. Press `Enter`, then `~` and `.` to force the frozen ssh session to close.

14. Confirm that the opening only applies to the client's ip address. While the port is open for the client (resend the SPA packet if needed), try to ssh into the server from a **different** machine, such as a classmate's machine or a third virtual machine.

```
ssh -p 22 username@138.47.x.x   # the server's ip address
```

It should not allow the other machine in, since the `SOURCE` in `access.conf` only allows the client's ip address, and the temporary rule fwknopd adds only matches that ip address.

15. On the server, to prevent the client from being locked out when the time limit expires, let's modify the firewall rules so that connections that were already established are allowed to continue.

    i. First, delete the rule we already have.

```
sudo iptables -D INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
```

You can confirm that it was deleted with `sudo iptables -L -nv`.

    ii. Next, add the following rule to allow related and established connections. The `conntrack` module tracks the state of each connection, so packets belonging to a connection that was already accepted are allowed through.

```
sudo iptables -A INPUT -i $INTERFACE -p tcp --dport $SSH_PORT \
    -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

iii. Re-add the rule to drop all ssh connections.

```
sudo iptables -A INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
```

iv. View the rules again with `sudo iptables -L -nv`. The INPUT chain should now contain, in order: the jump to FWKNOP_INPUT, the ACCEPT rule for established connections, and the DROP rule. Order matters, since iptables uses the first rule that matches.

16. Test the new rules.

i. On the client, send a new SPA packet and connect via ssh.

```
fwknop -n $SERVER_IP
ssh -p $SSH_PORT username@$SERVER_IP
```

ii. Stay connected until the time limit has expired, then run a few commands. The session should keep working this time.

iii. In a second terminal on the client, try to open a new ssh connection without sending a new SPA packet. It should not be allowed in, since new connections still require an SPA packet.

iv. Disconnect from the server.

17. (Optional) Switch SSH to key-based authentication. SPA only opens the door; the client still has to log in to SSH, and an SSH key is much stronger than a password.

i. On the client, generate an SSH key pair. Press `Enter` to accept the default location, and choose a passphrase.

```
ssh-keygen -t ed25519
```

ii. On the client, send an SPA packet, then copy the public key to the server.

```
fwknop -n $SERVER_IP
ssh-copy-id -p $SSH_PORT username@$SERVER_IP
```

iii. On the client, send another SPA packet and log in. You should be asked for the key's passphrase instead of the server's password.

```
fwknop -n $SERVER_IP
ssh -p $SSH_PORT username@$SERVER_IP
```

Now an attacker would need your SPA keys **and** your SSH private key to get in. This is defense in depth.

3 Going Further (Optional)

Take a look at what an SPA packet actually looks like on the wire. On the server, use `tcpdump` to capture packets sent to fwknop's default port (UDP 62201) and print their contents.

```
sudo tcpdump -i $INTERFACE -X udp port 62201
```

Then, on the client, send an SPA packet with `fwknop -n $SERVER_IP`. Notice that the payload is unreadable, since it is encrypted. Also notice that nothing on the server is listening on port 62201. You can confirm this with `sudo ss -aun` on the server (port 62201 will not appear), or with `sudo nmap -sU -p 62201 $SERVER_IP` on the client, which should report the port as `closed`. The port is closed rather than filtered because no firewall rule drops that traffic, so the server's kernel replies that nothing is listening there. Yet fwknopd still receives the packet, because it passively sniffs the traffic instead of accepting connections. There is nothing for an attacker to find with a port scan.

Stop tcpdump with `ctrl+c`.

4 Clean Up

Once you've completed the lab, reset the server back to how it was before.

1. On the server, make sure the variables are still set. If either `echo` prints a blank line, set it again (replace `enp0s3` with your interface).

   ```
   echo $SSH_PORT
   echo $INTERFACE
   # if blank:
   SSH_PORT=22
   INTERFACE=enp0s3
   ```

2. On the server, delete the two rules you added to the INPUT chain.

```
sudo iptables -D INPUT -i $INTERFACE -p tcp --dport $SSH_PORT \
    -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
sudo iptables -D INPUT -i $INTERFACE -p tcp --dport $SSH_PORT -j DROP
```

3. On the server, stop the fwknop server and prevent it from starting on boot. When fwknopd stops, it removes its own FWKNOP_INPUT chain.

```
sudo systemctl stop fwknop-server
sudo systemctl disable fwknop-server
```

4. On the server, confirm that the INPUT chain no longer contains any of the rules from this lab.

```
sudo iptables -L -nv
```

5. Remove the SPA keys so they are not left on either machine.

   i. On the server, open `/etc/fwknop/access.conf` and change the `KEY_BASE64` and `HMAC_KEY_BASE64` values back to `__CHANGE_ME__` (and `SOURCE` back to `ANY`), or delete the `OPEN_PORTS` line and the key lines you added.

   ii. On the client, delete the saved stanza and keys.

```
rm ~/.fwknoprc
```

6. (If you completed the optional key-based authentication step) On the server, remove the client's key from `~/.ssh/authorized_keys` if you do not want to keep it.

Note that rules added with `iptables` are not saved, so they are also cleared when the machine reboots. If you ever lock yourself out of a machine while experimenting with firewall rules, restarting it will restore access (as long as you have not made the rules persistent).